Defect Log & Analysis Report

IBA Campus Facility Booking System — QA Artifact

Prepared by Group 4:

Abdul Wasay Imran (27126) Saad Imam (27079)

Muhammad Imad Raza (26953) Syed Muhammad Deebaj Haider Kazmi (26012)

Version 1.0 — May 28, 2026

1 Introduction

1.1 Purpose

This document serves as the official Quality Assurance Defect Log for the IBA Campus Facility Booking System. It lists all issues discovered during the execution of system, integration, and end-to-end (E2E) automation test runs. This ledger acts as a key tracking mechanism to bridge communication between the QA and development teams, ensuring every gap identified against the Software Requirements Specification (SRS) is addressed prior to release.

1.2 Defect Classification System

Defects are assigned severity levels in accordance with the criteria defined within the Project Test Plan:

- **S1 (Critical):** System crash, database corruption, role-based security vulnerability, or functional blocker with no possible workaround.
- **S2 (Major):** Core feature failure (such as cancellation or valid booking prevention) where no viable workaround exists for the user.
- **S3 (Minor):** Functional or logical deviation that does not block core workflows; minor manual workarounds or simple configuration adjustments exist.
- **S4 (Trivial):** Cosmetic issues, UI misalignments, spelling mistakes, or minor non-blocking dashboard formatting.

1.3 Overall Defect Matrix Summary

The dynamic test runs revealed a total of **15 defects**. The distribution of severity levels across the discovered issues highlights critical database constraints and logical issues that are detailed later in this report:

Total Defects	S1 (Critical)	S2 (Major)	S3 (Minor)	S4 (Trivial)
15	4	9	2	0

2 Defect Ledger Summary Table

To prevent layout spillover, this master summary table is formatted using autowrapping dynamic columns.

Table 1: Master Defect Index

ID	TC Ref.	Sev.	Defect Summary Title	Status
DEF-001	TC-CANCEL-001/2/5	S2	Cancelled bookings are incorrectly marked as 'rejected'	Open
DEF-002	TC-BOOK/CANCEL-BUG	S1	Unconditional unique DB constraint blocks slot re-booking	Open
DEF-003	TC-BOOK-PAST	S2	Missing validation allows users to book rooms for past dates	Open
DEF-004	TC-CANCEL-002	S2	Student dashboard lacks 'Cancel' button on approved slots	Open
DEF-005	TC-BOOK-008	S2	Missing boundary check allows multi-year bookings	Open
DEF-006	TC-BOOK-004/5	S3	Time slots in schema violate business hours of 9 AM - 6 PM	Open
DEF-007	TC-CANCEL-004	S2	Missing boundary check allows cancellation of elapsed bookings	Open
DEF-008	TC-ADMIN-006	S2	'Disable Room' implemented only as single slot blocking	Open
DEF-009	TC-AUTH-001	S3	Missing 'Faculty/Teacher' user role	Open
DEF-010	TC-BOOK-010	S1	Missing Range Validation on `slot_id` crashes Database	Open
DEF-011	TC-DATA-002	S1	Cascade delete wipes out historical booking audit trails	Open
DEF-012	TC-DATA-001	S2	Student cancel overwrites PO reference in `reviewed_by`	Open
DEF-013	TC-PO-001	S2	GET /bookings fetches all database records without pagination	Open
DEF-014	TC-SEC-001	S2	Room Booking purpose input is vulnerable to stored XSS	Open
DEF-015	TC-DATA-001	S2	API SELECT explicitly omits `reviewed_by` field	Open

3 Detailed Defect Index (2 per Page)

DEF-001: Cancelled bookings are incorrectly marked as 'rejected'

Reference TC: TC-CANCEL-001/002/005 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Authenticate as a student. 2. Create a pending room booking. 3. Send a `PATCH` to `/api/bookings/:id/cancel`. 4. Observe response.
- **Expected:** Booking status field is updated to `"cancelled"`.
- **Actual:** Booking status field returned is `"rejected"`.
- **Root Cause:** In `bookings.service.ts`, the cancel controller hardcodes target status payload updates:

```
    async cancel(id: string, requesterId: string, requesterRole: string) {
        return this.updateStatus(id, 'rejected', requesterId); // Hardcoded
    }

```

DEF-002: Unconditional DB unique constraint prevents slot recycling

Reference TC: TC-BOOK-BUG / TC-CANCEL-BUG **Severity:** S1 (Critical) **Status:** Open

---

- **Steps to Reproduce:** 1. Book Room A on slot 1. 2. Cancel or reject the booking. 3. Attempt to book the same room, date, and slot.
- **Expected:** System accepts the booking (`201 Created`) because the slot is now free.
- **Actual:** Database throws unhandled unique index violation, returning a server-side `500 Error`.
- **Root Cause:** The schema definitions have an unconditional database table-level unique constraint: `UNIQUE(room_id, date, slot_id)`. This index includes inactive rows. It must be a conditional index targeting active rows:

```
    CREATE UNIQUE INDEX idx_active_bookings ON bookings (room_id, date,
    slot_id) WHERE (status IN ('pending', 'approved'));

```

DEF-003: Missing validation checks allow room bookings for past dates

Reference TC: TC-BOOK-PAST **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in as a student. 2. Post a booking payload containing historical dates (e.g., `"2020-01-01"`).
- **Expected:** Request rejected with a `400 Bad Request` validation error block.
- **Actual:** System accepts the booking and creates a record in the database (`201 Created`).
- **Root Cause:** `CreateBookingDto` checks only if `date` is a string. There are no application-level checks or custom decorators (e.g., `@IsFutureDate()`) in the validation layer to block past inputs.

DEF-004: Approved bookings on student dashboard lack Cancel option

Reference TC: TC-CANCEL-002 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in as a student. 2. View approved bookings on dashboard cards. 3. Look for a cancellation trigger button.
- **Expected:** A cancellation button should display, as SRS 2.9 permits cancellation before the scheduled start time.
- **Actual:** Cancel options only display for cards matching `"pending"`.
- **Root Cause:** Inside the UI view layer (`StudentDashboard.jsx`), conditionally rendered control expressions are restricted using strict identity checks against `"pending"`, omitting `"approved"`.

DEF-005: Missing current-year constraint checks allow future reservations

Reference TC: TC-BOOK-008 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in as a student. 2. Submit booking request with date parameter set to `"2028-12-25"`.
- **Expected:** Request rejected with validation errors, since SRS 2.2.3 limits bookings to the current year.
- **Actual:** Request accepted (`201 Created`).
- **Root Cause:** The client UI enforces constraints using local date ranges, but the API schema (`CreateBookingDto`) lacks logical constraints to compare incoming years with the current server year.

DEF-006: Time slot definitions violate SRS-defined business hours

Reference TC: TC-BOOK-004/005 **Severity:** S3 (Minor) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in to the booking portal. 2. View dropdown options or query `/api/time-slots`.
- **Expected:** The available time slots should exist within the SRS 2.2.3 window of 9 AM to 6 PM.
- **Actual:** First slot starts at 8:30 AM, and the last slot ends at 6:45 PM.
- **Root Cause:** Static row values in `init.sql` violate the boundaries defined in the SRS:

```
    -- Violators:
    (1, '08:30', '09:45', '8:30 - 9:45'),
    (7, '17:30', '18:45', '5:30 - 6:45');

```

DEF-007: Lack of temporal check allows cancellation of elapsed bookings

Reference TC: TC-CANCEL-004 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Identify an approved booking from last week. 2. Send cancellation request to endpoint.
- **Expected:** Cancellation fails with `400 Bad Request` because the booking has already passed.
- **Actual:** Request processes successfully and status updates to cancelled (`200 OK`).
- **Root Cause:** The backend `cancel()` method inside `bookings.service.ts` verifies row ownership and current status, but does not compare system times with the slot date.

DEF-008: Disable Room requirement implemented only as slot blocks

Reference TC: TC-ADMIN-006 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in as an admin. 2. Try to disable a room indefinitely for maintenance.
- **Expected:** Admins can toggle a room status to inactive to block all bookings, per SRS 2.6.
- **Actual:** Admins must manually block every individual time slot using `BlockedSlots`.
- **Root Cause:** Database schema contains no `is_active` or status flags on the `rooms` table.

DEF-009: Missing 'Faculty/Teacher' user role in system schemas

Reference TC: TC-AUTH-001 **Severity:** S3 (Minor) **Status:** Open

---

- **Steps to Reproduce:** 1. Attempt to register or configure an account with a role key of `faculty` or `teacher`.
- **Expected:** System registers account, per SRS Objective 6 (distinguishing student and faculty applicants).
- **Actual:** API returns an unhandled database error block.
- **Root Cause:** Database role configurations do not include a faculty entry:

```
    CREATE TYPE user_role AS ENUM('student', 'programoffice', 'admin'); --
    Missing 'faculty'

```

DEF-010: Missing range validation on slot ID crashes the database

Reference TC: TC-BOOK-010 **Severity:** S1 (Critical) **Status:** Open

---

- **Steps to Reproduce:** 1. Authenticate as a student. 2. Send booking request with an invalid `slot_id` (e.g., 9999).
- **Expected:** Request rejected with a validation error (`400 Bad Request`).
- **Actual:** Database throws a foreign key constraint violation, causing an unhandled `500 Error`.
- **Root Cause:** `CreateBookingDto` validates types using only `@IsInt()`. It lacks index range limits:

```
    // Fix:
    @IsInt() @Min(1) @Max(7)
    slot_id: number;

```

DEF-011: Cascade deletes destroy historical audit trails

Reference TC: TC-DATA-002 **Severity:** S1 (Critical) **Status:** Open

---

- **Steps to Reproduce:** 1. Identify a user with bookings. 2. As an Admin, run a delete call to `/api/users/:id`.
- **Expected:** User deletion blocks, soft-deletes, or nullifies the field to preserve booking history.
- **Actual:** All associated booking history records are permanently deleted.
- **Root Cause:** Foreign key constraints on the backend database are configured with cascade deletes:

```
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE -- Wipes
booking history

```

DEF-012: Student cancellation overwrites reviewer ID values

Reference TC: TC-DATA-001 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Book a room. 2. Have the PO approve it. 3. Cancel booking as student. 4. Query DB record.
- **Expected:** `reviewed_by` retains the PO UUID, while a separate cancellation actor field updates.
- **Actual:** `reviewed_by` is overwritten with the student's ID.
- **Root Cause:** Inside `bookings.service.ts`, the student cancel handler routes to updateStatus:

```
    // Student UUID passes as the reviewer parameter:
    return this.updateStatus(id, 'rejected', requesterId);

```

DEF-013: Missing pagination on database query returns entire booking history

Reference TC: TC-PO-001 / TC-PERF-001 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Seed database with 5,000+ entries. 2. Access PO portal. 3. Monitor payload scale and rendering lag.
- **Expected:** System requests and returns targeted page segments (e.g., limit of 50).
- **Actual:** Server yields massive array blocks containing every record, slowing page performance.
- **Root Cause:** Database retrieval logic in `bookings.service.ts` uses simple select scans:

```
1     this.supabase.db.from('bookings').select(SELECT) // Lacks page size/limit
       constraints
2
```

DEF-014: Room Booking purpose input field vulnerable to stored XSS

Reference TC: TC-SEC-001 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Log in as a student. 2. Submit purpose field containing markup: `<script>alert('xss')</script>`.
- **Expected:** Request is sanitized or rejected before storage.
- **Actual:** API accepts and stores payload tags. (If the frontend does not safely escape it, it can execute).
- **Root Cause:** `CreateBookingDto` validates inputs as standard strings via `@IsString()`, but performs no sanitization or tag filtering before writing to the database.

DEF-015: API SELECT statements explicitly omit reviewer ID field

Reference TC: TC-DATA-001 **Severity:** S2 (Major) **Status:** Open

---

- **Steps to Reproduce:** 1. Approve a booking. 2. Send a `GET` request to `/api/bookings` or `/api/bookings/:id`.
- **Expected:** Returned response payload contains the `reviewed_by` field to trace approval history.
- **Actual:** The field is omitted, breaking the frontend audit trail.
- **Root Cause:** Inside `bookings.service.ts`, the select query projection explicitly excludes the `reviewed_by` field, making it impossible for the UI to display which PO member authorized a request:

```
    // In bookings.service.ts:
    const SELECT = 'id, user_id, room_id, slot_id, date, purpose, status'; // Missing reviewed_by

```

— End of Defect Log Report —